

RHEL Image Mode / bootc-demo

Talernoter og demo-manuskript

Indledning

I dag vil jeg vise en anden måde at tænke Linux-livscyklusstyring på.

I stedet for at installere en server og løbende ændre den over tid, definerer vi operativsystemet som et image. Dette image bygges fra kildekode, gemmes i et registry og fungerer som livscyklusartefakt for systemet.

Det vigtigste er ikke selve `bootc`-kommandoen. Det vigtige er den driftsmodel, der bygges omkring den.

Hovedbudskab

Operativsystemet bliver et versionsstyret, reproducerbart artefakt, der kan rulles tilbage.

Hvorfor er det vigtigt?

Traditionel serveradministration skaber ofte:

- Konfigurationsdrift
- Manuel patching og vedligeholdelsesvinduer
- Uensartede miljøer
- Begrænsede muligheder for rollback
- Dårlig sporbarhed fra ændringsanmodning til kørende system

Image Mode flytter dette i retning af:

- Git-baseret ændringsstyring
- Reproducerbare builds
- Registry-baseret distribution
- Transaktionelle opdateringer
- Operationel rollback
- Bedre standardisering

Hvorfor er immutable images vigtige?

Et immutable image er en defineret version af operativsystemet. Vi behandler ikke den kørende host som et sted, hvor man foretager ukontrollerede, permanente ændringer. Når systemet skal ændres, bygger og udruller vi i stedet en ny image-version.

Containere følger samme model: Imaget bygges én gang, promoveres gennem pipeline'en og kører ensartet i alle miljøer. Containeren eller VM'en kan udskiftes, mens imaget forbliver den sporbare kilde til systemets tilstand.

Fordele

- **Ensartethed:** Udvikling, test og produktion bruger det samme byggede artefakt.
- **Mindre drift:** Ændringer foretages i kilden og bygges igen, frem for at blive udført forskelligt på individuelle systemer.
- **Sikrere drift:** Opdateringer kan testes og valideres før udrulning.
- **Hurtig gendannelse:** Fejlslagne opdateringer kan rulles tilbage til et kendt fungerende image.
- **Sporbarhed:** Det kørende system kan kobles til image-digest, Git-commit og pipeline-kørsel.

Immutable betyder ikke, at systemet aldrig kan ændres. Det betyder, at ændringerne er tilsigtede, versionsstyrede og leveres gennem en kontrolleret livscyklus.

Demo-trin 1: Repository

Det siger jeg

Dette repository er sandhedskilden.

- Containerfile definerer operativsystemets tilstand.
- Scripts automatiserer lokale operationer.
- GitHub Actions bygger imaget.
- Quay fungerer som distributionspunkt.

Hvorfor er det vigtigt?

Alle ændringer kan gennemgås, godkendes og auditeres, før de bliver til kørende infrastruktur.

Demo-trin 2: Containerfile

Det siger jeg

Containerfile beskriver den ønskede tilstand for operativsystemet. Det kan omfatte pakker, filer, services og konfiguration.

Den afgørende forskel er, at Containerfile beskriver det ønskede image, mens det kørende system blot er en udrullet version af imaget.

Kundeværdi

- Standardiserede OS-baselines
- Golden Images
- Ensartede udrulningsmønstre

Demo-trin 3: Build

Det siger jeg

Nu bygger vi operativsystem-imaget. Det minder om den måde application teams bygger container images på, men målet er et bootbart operativsystem.

Kundeværdi

De samme governance- og kvalitetsmekanismer, som kunder allerede bruger til applikationer, kan nu bruges på operativsystemlaget. Imaget bliver det immutable bindeled mellem build og runtime. Når det er bygget og testet, kan det samme artefakt promoveres gennem miljøerne.

Demo-trin 4: Quay

Det siger jeg

Imaget publiceres til Quay. Operativsystemets livscyklus er nu knyttet til et registry-artefakt med tags, historik, signering og scanning.

Kundeværdi

Registry'et bliver et kontrolleret distributionspunkt for standardiserede operativsystem-images.

Demo-trin 5: QCOW2 / VM

Det siger jeg

For at gøre resultatet konkret konverterer vi bootc-imaget til en VM-disk og starter den.

Kundeværdi

Det forbinder moderne image-baseret livscyklusstyring med eksisterende virtualiseringsplatforme.

Demo-trin 6: bootc status

Det siger jeg

Det kørende system ved præcist, hvilket image det er baseret på. Det skaber en tydelig forbindelse mellem kildekode, image og kørende host.

Kundeværdi

Forbedret sporbarhed og lettere support.

Demo-trin 7: Opdatering

Det siger jeg

I stedet for manuelt at patche individuelle pakker flytter vi systemet til en ny image-version. Vi erstatter systemet med en ny deklareret version frem for at akkumulere et ukendt sæt ændringer på den eksisterende host. Det er her immutability reducerer driftskompleksitet over tid.

Kundeværdi

- Mere forudsigelig ændringshåndtering
- Bedre testmuligheder
- Mere kontrollerede produktionsudrulninger

Demo-trin 8: Rollback

Det siger jeg

Hvis den nye tilstand ikke fungerer, ruller vi tilbage til det tidligere kendte, velfungerende image. Fordi det tidligere image stadig eksisterer som en verificeret version, afhænger gendannelse ikke af, hvilke pakker eller konfigurationsfiler der blev ændret.

Kundeværdi

Rollback bliver hurtigt, forudsigeligt og operationelt simpelt.

Vinkling til forskellige målgrupper

Linux-drift

- Standardiserede baselines
- Ensartet patching
- Mindre drift
- Hurtigere gendannelse

OpenShift- og Kubernetes-teams

- GitOps-tankegang
- Immutable infrastruktur
- Platform Engineering
- OpenShift Virtualization

Enterprise-arkitekter

- Governance
- Sporbarhed
- Software Supply Chain
- Livscyklusstyring

Ledelse

- Lavere operationel risiko
- Mere forudsigelige ændringer
- Hurtigere rollback
- Tydeligt ejerskab

Stærk afslutning

Denne demo handler ikke om at erstatte alle Linux-administrationsprocesser i morgen. Den handler om at vise, hvor Linux-livscyklusstyring bevæger sig hen:

- Versionsstyrede images
- Kontrollerede pipelines
- Registry-baseret distribution
- Sikrere Day 2-drift

For mange kunder er første skridt ikke en produktionsudrulning. Det første skridt er at vurdere:

Hvor kan image-baseret livscyklusstyring reducere drift, forbedre compliance og forenkle platformdrift?

Afslutning til arkitekter eller ledelse:

Vi flytter operativsystemet fra at være noget, der administreres manuelt, til noget, der leveres og styres som software.